DIGITAL NOTES • SOFT COMPUTING

UNIT – IV

Genetic Algorithms

What is inside this unit

1	Basic concepts and terminology, encoding
2	Genetic algorithm cycle (flowchart, termination, replacement)
3	Fitness function and scaling
4	Basic operators – selection, crossover, mutation
5	Crossover and mutation properties, schema theorem
6	Worked numerical example
7	Applications of GA
8	Revision sheet & exam questions

After this unit you will be able to

- Explain GA terminology and encode a problem.
- Describe the GA cycle and write its algorithm.
- Design a fitness function and choose a selection method.
- Apply crossover and mutation and discuss pc and pm.
- Solve a one-generation GA example by hand.

How to use these notes

Read the **blue headings** first, study each diagram, then read the points. Yellow boxes = exam tips / key formulas. Green boxes = quick memory tricks. The last page has a one-page revision sheet and likely exam questions.

1 Genetic Algorithms – Basic Concepts

A **Genetic Algorithm (GA)** is a **search and optimisation technique based on natural selection and genetics**, introduced by **John Holland (1975)** and popularised by D. E. Goldberg. It evolves a population of candidate solutions over generations; **fitter solutions get more chances to reproduce**, so the population steadily improves.

- **Idea:** "survival of the fittest" + "inheritance of good genes" = search for the best solution.
- **Population-based:** works with many solutions at once, not just one point.
- **Uses probabilistic rules** (random selection, crossover, mutation) – not deterministic ones.
- **Needs only fitness values** – no derivatives or smoothness required.

1.1 Biological terms used in GA

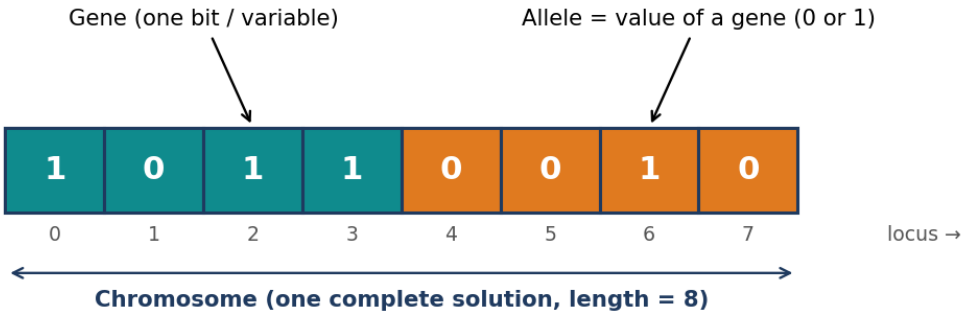


Fig 1.1 Anatomy of a chromosome

Term	Biology	In GA
Gene	Unit of heredity	One position (variable / bit) in the string
Allele	Value of a gene	0 or 1 (or a number / symbol)
Chromosome (individual)	DNA strand	One complete candidate solution (string)
Locus	Position of a gene	Index in the string
Population	Group of organisms	Set of chromosomes (size N)
Genotype	Genetic make-up	Encoded string, e.g. 01101
Phenotype	Observable body	Decoded solution, e.g. x = 13
Fitness	Survival ability	Quality of the solution (objective value)
Generation	One breeding cycle	One iteration of the GA

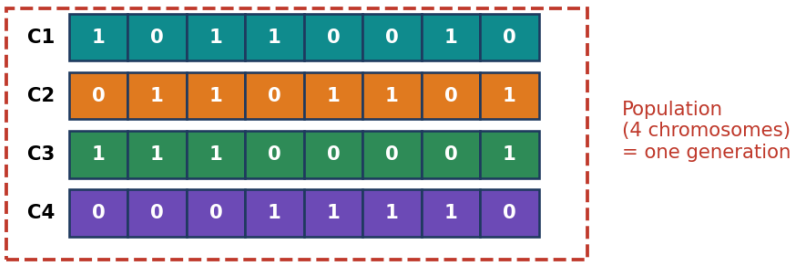


Fig 1.2 A population of four chromosomes

1.2 Encoding (representation) of solutions

The problem's variables must first be **encoded** as chromosomes. Good encoding makes crossover and mutation produce **valid** solutions.

Encoding	Example	Used for
Binary	1 0 1 1 0 → x = 22	Numerical optimisation, knapsack, feature selection
Value (real)	2.5 7.1 0.3	Continuous parameters, NN weights
Permutation	3 1 4 2 5	Travelling salesman, scheduling (order matters)
Tree	(+ (× x 2) y)	Genetic programming – evolving programs / formulas

Decoding a binary gene

For a variable in [a, b] using n bits: **$x = a + (b - a) \times$ (decimal value) / $(2^n - 1)$**. Example: n = 5, range [0, 31] → string 01101 = 13 → x = 13.

2 Genetic Algorithm Cycle

The GA repeats the same loop – **evaluate → select → crossover → mutate** – until a stopping rule is met.

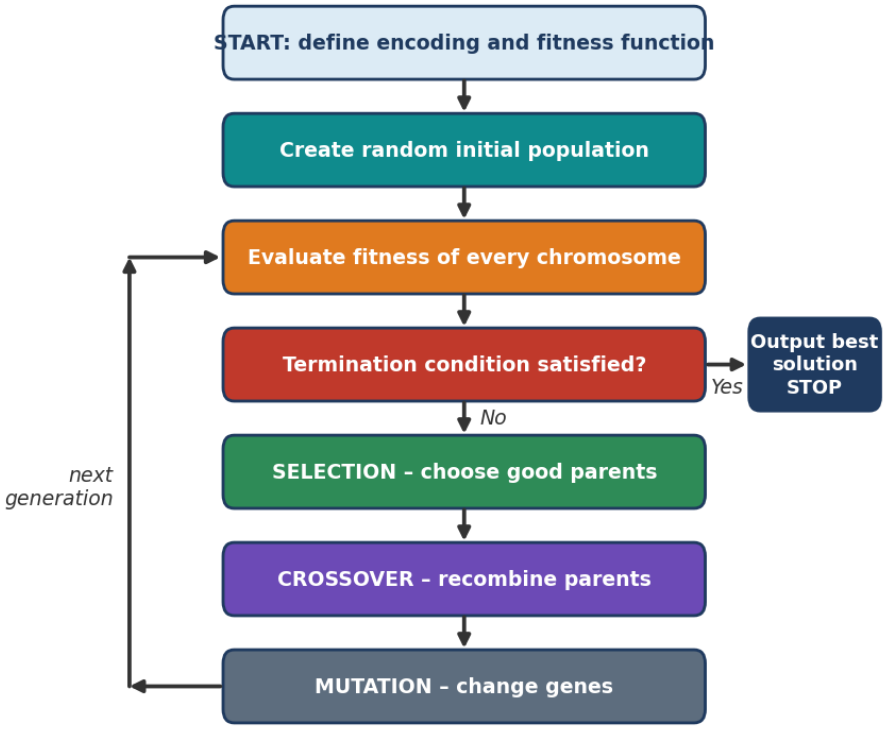


Fig 2.1 Flow chart of the genetic algorithm cycle

Genetic Algorithm
1. Choose encoding, population size N, pc, pm; generation t = 0
2. Initialise population P(0) randomly
3. Evaluate fitness of all individuals
4. **while** (stopping criterion not met):
   a) select parents (by fitness)
   b) crossover parents with probability pc → offspring
   c) mutate offspring genes with probability pm
   d) evaluate offspring; build P(t+1); t = t + 1
5. return the best chromosome found

2.1 Stopping (termination) criteria

- Fixed maximum number of generations reached.
- Fitness target achieved (good enough solution).
- No improvement in best fitness for many generations (convergence).
- Time or computing budget finished.

2.2 Replacement strategies

- **Generational:** the whole population is replaced by offspring.
- **Steady-state:** only a few worst members are replaced each time.
- **Elitism:** the best 1–2 chromosomes are copied unchanged to the next generation, so the best solution is never lost.

3 Fitness Function

The **fitness function** is the "judge" of the GA: it gives every chromosome a score showing **how good the solution is**. It is the only link between the GA and the real problem.

- **Maximisation:** fitness = objective value, e.g. $f(x) = x^2$.
- **Minimisation:** convert, e.g. fitness = 1 / (1 + cost) or C_{max} − cost.
- **Constraints:** subtract a **penalty** for violating a constraint: fitness = f(x) − penalty.

Qualities of a good fitness function

• **Correct** – higher score really means better solution. • **Fast to compute** – it is called thousands of times. • **Discriminating** – gives different scores to different quality solutions. • **Smooth enough** – small change in gene gives small change in score. • **Non-negative** – needed by roulette-wheel selection.

3.1 Fitness landscape

Imagine fitness as hills: the highest peak is the global optimum, smaller peaks are local optima. A GA starts with points scattered everywhere and gradually gathers them near the highest peaks.

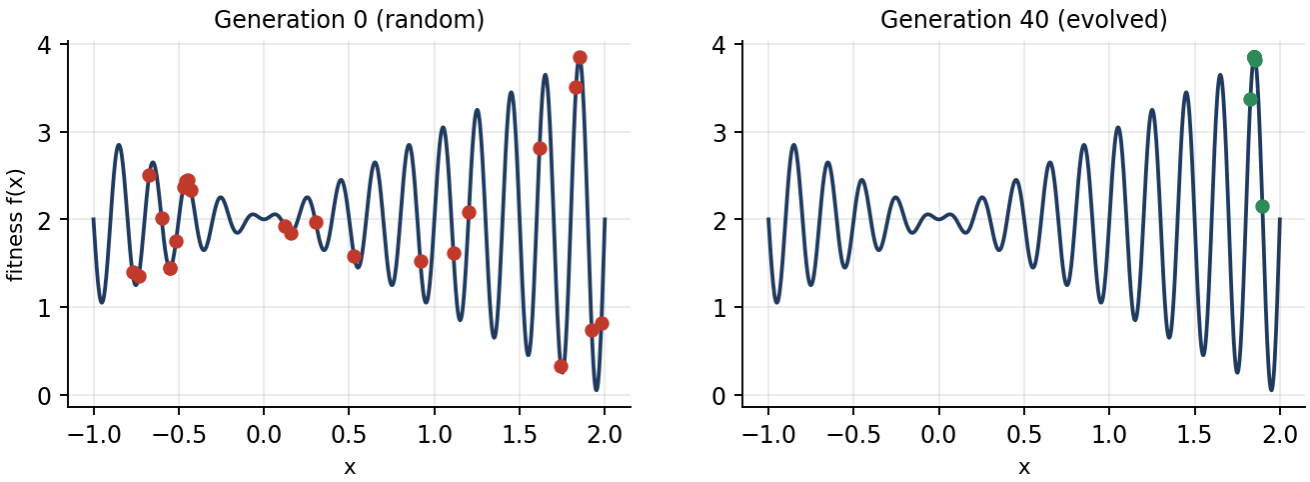


Fig 3.1 GA on f(x) = x·sin(10πx) + 2 : random start vs after 40 generations (simulation)

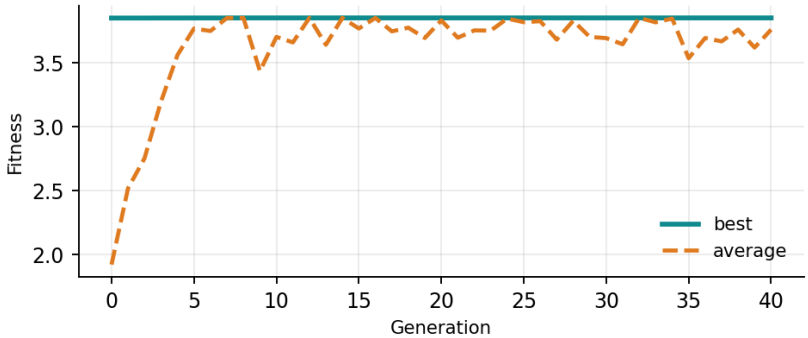


Fig 3.2 Convergence – best and average fitness rise and come together

3.2 Fitness scaling

- **Problem:** one super-fit chromosome early on dominates selection → premature convergence.
- **Linear scaling:** $f' = a \cdot f + b$ keeps average fitness fixed and limits the best to 1.5–2× the average.
- **Rank-based:** use the rank instead of raw fitness, so differences in magnitude do not matter.

4 Basic Operators of Genetic Algorithms

Three operators drive the GA: **Selection** (exploit good solutions), **Crossover** (combine them) and **Mutation** (explore new ones).

4.1 Selection (reproduction)

Chooses parents so that **better chromosomes are picked more often**, yet weaker ones keep a small chance (diversity).

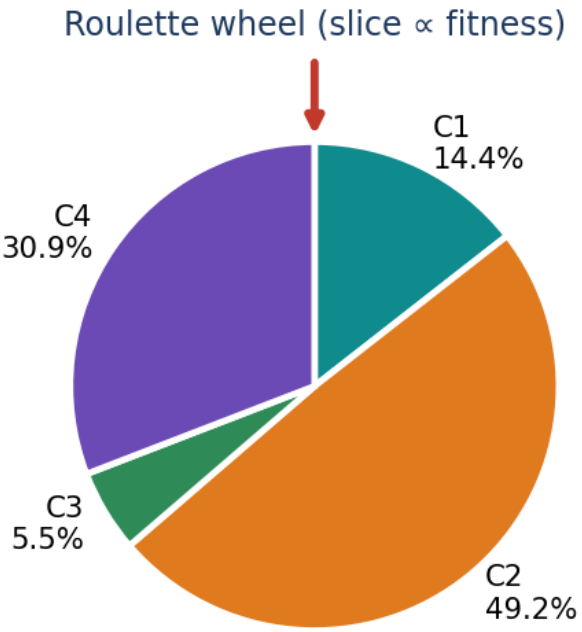


Fig 4.1 Roulette wheel selection for the example population (fitness 169, 576, 64, 361)

Method	How it works	Remark
Roulette wheel	Slice size ∝ fitness; spin N times. $p_i = f_i / \Sigma f$	Simple; fails if one fitness is huge
Tournament	Pick k random individuals; the best wins	Popular, no scaling needed, k = 2–5 sets pressure
Rank	Sort by fitness, probability by rank	Avoids domination
Stochastic universal sampling	One spin, N equally spaced pointers	Lower variance than roulette
Elitism	Copy best 1–2 directly	Never loses the best

4.2 Crossover (recombination)

Takes **two parents** and exchanges parts of their strings to create **two children**. It is the main search operator and is applied with probability **pc**.



Fig 4.2 Single-point crossover

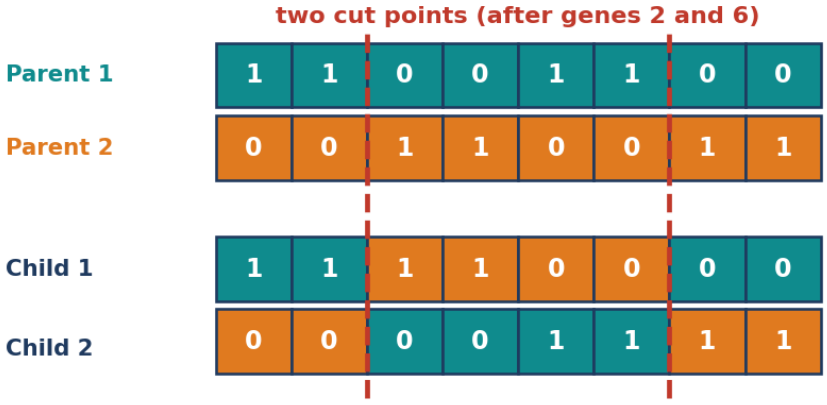


Fig 4.3 Two-point crossover

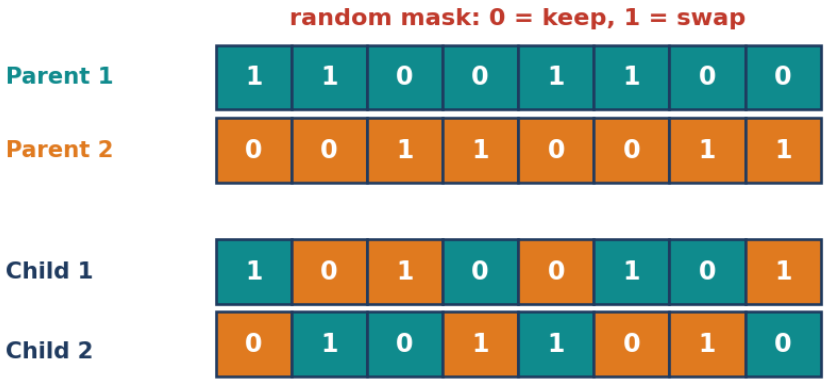


Fig 4.4 Uniform crossover (a random mask decides the source of every gene)

Type	Remark
Single-point	Simplest; one cut point; may break long building blocks
Two-point / k-point	Swaps middle segment; less positional bias
Uniform	Each gene from either parent (p = 0.5); high mixing
Arithmetic (real genes)	child = α·p1 + (1 − α)·p2
PMX / Order (OX)	For permutations – keep every city exactly once (TSP)

4.3 Mutation

Randomly changes genes with a **small probability pm**. It restores lost genes and adds **diversity**, so the GA can escape local optima.

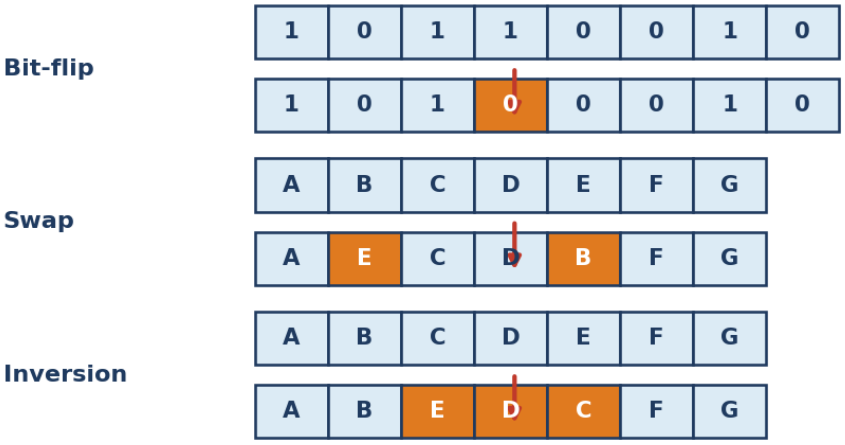


Fig 4.5 Mutation types: bit-flip (binary), swap and inversion (permutation)

- **Bit-flip:** 0 ↔ 1 for each gene with prob. pm. **Random reset / Gaussian:** for real genes, add small random noise.
- **Swap / Inversion:** for permutation chromosomes (keeps the tour valid).

5 Crossover and Mutation Properties

Property	Crossover	Mutation
Role	**Exploitation** – combines good parts of parents	**Exploration** – introduces new genetic material
Parents needed	Two (or more)	One
Typical rate	pc = 0.6 – 0.9 (high)	pm = 0.001 – 0.05, about 1 / chromosome length (low)
Effect on population	Moves population towards good regions fast	Prevents premature convergence and loss of genes
If too high	Good parents are disrupted	GA becomes a random search
If too low	Slow improvement	Stuck in local optimum
Alone	Cannot create a gene not present in parents	Too slow to build good solutions by itself

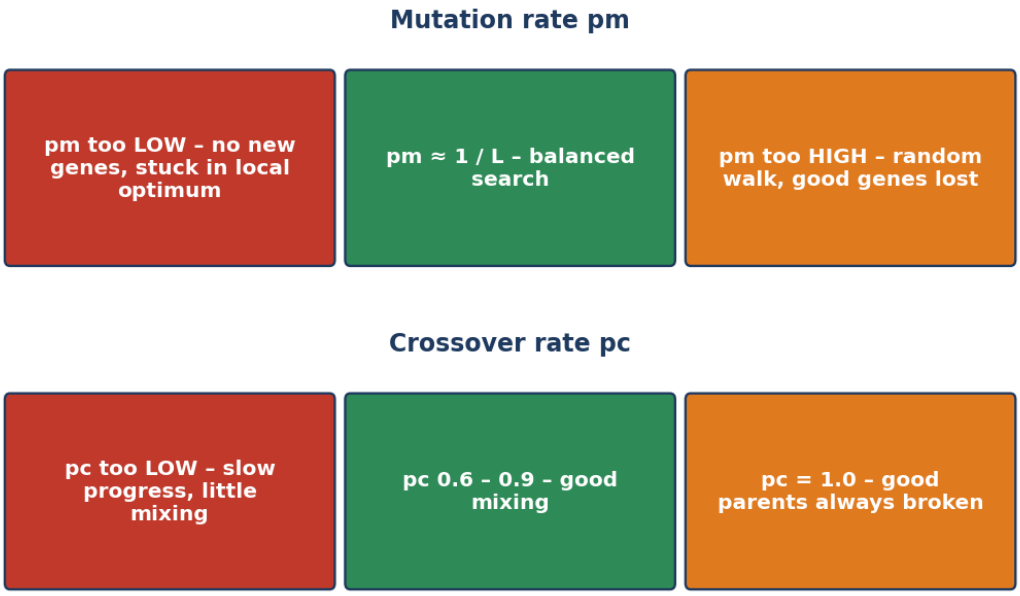


Fig 5.1 What happens when the rates are wrong

5.1 Why does a GA work? – Schema theorem

- A **schema** is a pattern with wildcards, e.g. **1 * * 0 *** (* = don't care). It stands for a whole family of strings.
- **Order** = number of fixed genes; **defining length** = distance between first and last fixed gene.
- **Schema theorem (Holland):** short, low-order, above-average schemata receive exponentially more copies in successive generations.
- These are the **building blocks**; crossover combines them into better and bigger solutions (**building block hypothesis**).

Memory trick

Crossover = "mix and match", Mutation = "small surprise". Together: crossover exploits, mutation explores.

6 Worked Example – Maximise f(x) = x²

Find x in [0, 31] that maximises f(x) = x². Use **5-bit binary** strings, population N = 4, roulette-wheel selection, single-point crossover.

Step 1 – Initial population and fitness

Chrom.	String	x	f(x) = x²	Probability f/Σf	Expected copies
C1	01101	13	169	0.144	0.58
C2	11000	24	576	0.492	1.97
C3	01000	8	64	0.055	0.22
C4	10011	19	361	0.309	1.23

Σf = 1170, average = 292.5, best = 576. **C2** (fittest) is expected to get ~2 copies, **C3** (weakest) ~0.

Step 2 – Selection (mating pool)

After roulette-wheel spins the mating pool is: **C1, C2, C2, C4** (C3 is dropped).

Step 3 – Crossover

Pair	Parents	Cut	Children	x values	f(x)
1	01101 , 11000	after gene 4	01100 , 11001	12 , 25	144 , 625
2	11000 , 10011	after gene 2	11011 , 10000	27 , 16	729 , 256

Step 4 – Mutation

With a small pm, suppose only the third gene of the last child flips: **10000 → 10100** (x = 20, f = 400).

Step 5 – New generation

	Strings	Σ f	Average f	Best f
Old	01101 11000 01000 10011	1170	292.5	576
New	01100 11001 11011 10100	1898	474.5	729

Result

In **one generation** the average fitness rose from 292.5 to 474.5 and the best from 576 to 729. Repeating the cycle drives the population towards **11111 (x = 31, f = 961)**, the true optimum.

7 Applications of Genetic Algorithms

Area	Problem solved with GA
Engineering design	Optimising aircraft wings, truss structures, antenna shapes, circuit layout (VLSI)
Scheduling & planning	Timetabling, job-shop scheduling, vehicle routing, travelling salesman
Machine learning	Feature selection, neural-network weights/architecture, tuning fuzzy rules (genetic-fuzzy)
Robotics & control	Path planning, controller parameter tuning, evolving robot behaviour
Finance & business	Portfolio optimisation, trading rules, supply-chain optimisation
Bioinformatics & medicine	Protein folding, gene-expression clustering, drug design, radiotherapy planning
Image & signal processing	Image segmentation, compression, filter design
Games & creative	Game AI, evolutionary art and music, level generation

7.1 Advantages and limitations

Advantages	Limitations
Needs no derivatives; works on discontinuous, noisy functions	No guarantee of the global optimum
Searches many points at once – good global search	Needs many fitness evaluations (slow if fitness is costly)
Handles many variables, constraints and multiple objectives	Choosing encoding and parameters needs experience
Easily combined with fuzzy, neural and other methods	Premature convergence if diversity is lost

7.2 GA vs traditional optimisation

Point	Traditional (calculus / gradient)	Genetic algorithm
Search	From a single point	From a population of points
Information used	Derivatives needed	Only fitness values
Rules	Deterministic	Probabilistic
Local optima	Easily trapped	Better chance to escape
Search space	Smooth, continuous	Any – discrete, noisy, discontinuous

8 One-page Revision & Important Questions

Formula and fact sheet

• Selection probability $p_i = f_i / \Sigma f$. Expected copies = $p_i \times N$. • Decode: $x = a + (b - a) \cdot D / (2^n - 1)$.
• GA cycle: Encode → Initial population → Fitness → Selection → Crossover → Mutation → New generation → Stop?
• pc high (0.6–0.9), pm low (≈ 1/L). • Elitism keeps the best. • Schema theorem: short, low-order, fit schemata multiply.
• Crossover = exploitation; mutation = exploration.

Quick self-check

Q1. Who introduced GA? → **John Holland, 1975**
Q2. Which operator brings new genes? → **Mutation**
Q3. What does a fitness function do? → **Scores quality of each chromosome**
Q4. Best encoding for TSP? → **Permutation**

Likely exam questions

- Explain the basic concepts and terminology of genetic algorithms.
- Draw and explain the GA cycle / flowchart.
- What is a fitness function? How is it designed? Explain fitness scaling.
- Explain selection methods (roulette, tournament, rank) with examples.
- Explain crossover and mutation types with diagrams; discuss their properties.
- Solve a numerical: one generation of GA for $f(x) = x^2$.
- List applications, advantages and limitations of GA.